



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Universidad de las Fuerzas Armadas ESPE

Carrera de Software

Desarrollo de Videojuegos

Investigación de Técnicas de Toma de Decisiones en IA para Videojuegos

Caso de Estudio: Halo

Grupo: 1

Integrantes:

Jordan Guaman

Mesias Mariscal

Anthony Morales

Denise Rea

NRC: 30817

Docente: Ing. Héctor Pinto

Sangolquí, Ecuador

Junio de 2026

G1

Índice

1. Introducción	4
2. Técnicas de Toma de Decisiones	4
2.1. Máquinas de Estados Finitos (FSM)	5
2.1.1. Definición y fundamentos	5
2.1.2. Ventajas y limitaciones	5
2.1.3. Evolución: HFSM (Máquinas de Estados Jerárquicas)	5
2.1.4. Aplicación en Halo	6
2.2. Árboles de Comportamiento (Behavior Trees)	6
2.2.1. Definición y fundamentos	6
2.2.2. Ventajas y limitaciones	7
2.2.3. Aplicación en Halo	7
2.3. Árboles de Decisión (Decision Trees)	7
2.3.1. Definición y fundamentos	7
2.3.2. Ventajas y limitaciones	8
2.4. Sistemas Basados en Reglas	8
2.4.1. Definición y fundamentos	8
2.4.2. Aplicación en videojuegos	9
2.5. GOAP (Goal-Oriented Action Planning)	9
2.5.1. Definición y fundamentos	9
2.5.2. Limitaciones	9
2.6. Lógica Difusa (Fuzzy Logic)	9
2.6.1. Definición y fundamentos	9
2.6.2. Aplicación	10
2.7. Sistemas de Utilidad (Utility AI)	10
2.7.1. Definición y fundamentos	10
2.7.2. Aplicación en Halo	10
2.8. Aprendizaje por Refuerzo (Reinforcement Learning)	10
2.8.1. Definición y fundamentos	10
2.8.2. Realidad en la industria	11
3. Comprensión Conceptual: Análisis Comparativo	11
3.1. Guía de selección de técnica	12
3.2. Análisis comparativo transversal	12

3.3. Arquitecturas Híbridas	13
4. Aplicación en Halo: Identificación de Técnicas por Personaje	14
4.1. Análisis detallado: Grunts y Elites	14
4.1.1. Grunt (Infantería básica)	14
4.1.2. Elite (Rango superior)	15
5. Análisis de Funcionamiento: Flujo de Decisión en Halo	15
5.1. Diagrama General del Flujo de Decisión	15
5.2. Fase 1: Percepción	15
5.3. Fase 2: Evaluación de Condiciones	16
5.4. Fase 3: Selección de Acción	16
5.5. Fase 4: Ejecución	17
5.6. Fase 5: Cambio de Estado	17
6. Conclusiones	18

1. Introducción

La inteligencia artificial en los videojuegos ha experimentado una evolución profunda desde sus inicios en la década de 1970, cuando los enemigos seguían patrones de movimiento predecibles como el comportamiento de los fantasmas en *Pac-Man*. En la actualidad, los NPCs (personajes no jugadores) son capaces de exhibir comportamientos emergentes, adaptativos y tácticos que simulan inteligencia y credibilidad. En el centro de esta transformación se encuentran las **técnicas de toma de decisiones**, que constituyen el *cerebro* de cada agente virtual, determinando cómo reaccionan al entorno, al jugador y a otros NPCs.

La elección de la técnica adecuada depende de múltiples factores: el tipo de personaje, la frecuencia de actualización requerida, el presupuesto computacional disponible y el nivel de imprevisibilidad deseado. No existe una técnica universalmente superior; cada una presenta ventajas y limitaciones que la hacen más adecuada para contextos específicos.

Este documento presenta una investigación exhaustiva de ocho técnicas fundamentales de toma de decisiones utilizadas en la industria del videojuego, organizadas desde las más simples y predecibles hasta las más complejas y emergentes:

- Máquinas de Estados Finitos (FSM) y su evolución jerárquica (HFSM)
- Árboles de Comportamiento (Behavior Trees)
- Árboles de Decisión (Decision Trees)
- Sistemas Basados en Reglas (Rule-Based Systems)
- Planificación de Acciones Orientada a Objetivos (GOAP)
- Lógica Difusa (Fuzzy Logic)
- Sistemas de Utilidad (Utility AI)
- Aprendizaje por Refuerzo (Reinforcement Learning)

Se analiza su aplicación práctica en el videojuego *Halo* como caso de estudio, identificando qué técnicas emplean los distintos personajes (Grunts, Elites, Jackals, Hunters, Flood) y explicando en detalle el flujo de decisión que siguen: desde la percepción del entorno hasta el cambio de estado interno, pasando por la evaluación de condiciones, la selección de acciones y la ejecución de comandos en los subsistemas del motor.

2. Técnicas de Toma de Decisiones

2.1. Máquinas de Estados Finitos (FSM)

2.1.1. Definición y fundamentos

Una **Máquina de Estados Finitos** es un modelo computacional donde un agente solo puede estar en un único estado predefinido a la vez. Las transiciones entre estados ocurren al cumplirse condiciones específicas, usualmente definidas por eventos del juego o cambios en variables del entorno.

Formalmente, una FSM se define como la tupla (S, E, T, s_0, F) donde:

- S : conjunto finito de estados (ej: PATRULLAR, ALERTA, ATACAR)
- E : conjunto de eventos o condiciones de entrada
- $T : S \times E \rightarrow S$: función de transición que determina el siguiente estado
- $s_0 \in S$: estado inicial
- $F \subseteq S$: conjunto de estados finales o de aceptación

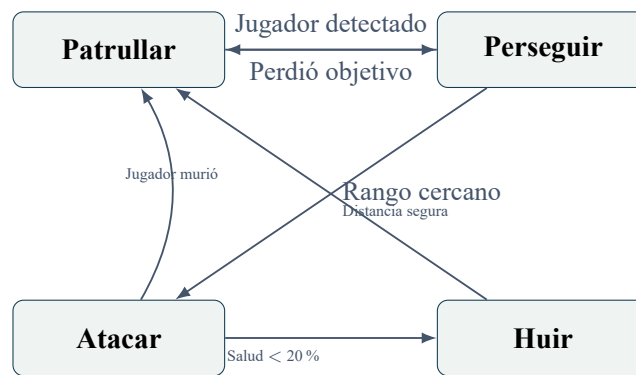


Figura 1: Diagrama de una Máquina de Estados Finitos para un NPC de patrulla. Cada estado representa un comportamiento y las flechas indican las condiciones de transición.

2.1.2. Ventajas y limitaciones

Las FSM son conceptualmente simples y fáciles de implementar, lo que las hace ideales para enemigos básicos con patrones predecibles. Sin embargo, su principal limitación es la *explosión de estados*: al crecer la lógica del comportamiento, el número de transiciones crece de forma cartesiana. Con n estados y m condiciones, se requieren hasta $n \times m$ transiciones potenciales, lo que genera código espagueti difícil de mantener y escalar.

2.1.3. Evolución: HFSM (Máquinas de Estados Jerárquicas)

La solución a la explosión de estados son las **HFSM**, que permiten anidar estados dentro de otros estados. Un estado *Combate* puede contener subestados como *Atacar*, *Esquivar* y *Recargar*, y

las transiciones de alto nivel (ej: *Recibir daño crítico*) interrumpen los subestados y llevan al agente a *Huir*. Esto reduce drásticamente el acoplamiento y mejora la organización del código.

2.1.4. Aplicación en Halo

Los **Flood** en Halo utilizan FSM puras con estados simples: *Merodear*, *Detectar*, *Cargar* y *Atacar cuerpo a cuerpo*. Este comportamiento de enjambre no requiere planificación compleja, solo reacciones rápidas y predecibles. Los **Hunters** combinan FSM con Behavior Trees: su estado base es de patrulla en pareja, pero al detectar al jugador transicionan a estados de disparo (ráfagas) y carga cuerpo a cuerpo, exponiendo su punto débil dorsal.

2.2. Árboles de Comportamiento (Behavior Trees)

2.2.1. Definición y fundamentos

Un **Árbol de Comportamiento** es una estructura jerárquica de nodos que se evalúa desde la raíz en cada *tick* del motor. A diferencia de las FSM, los BT no mantienen un estado explícito; en su lugar, evalúan todo el árbol en cada frame, lo que los hace inherentemente más reactivos.

Los tipos de nodos fundamentales son:

- **Raíz:** punto de entrada; ejecuta un único hijo en cada tick
- **Selector (Fallback):** ejecuta hijos en orden; devuelve Éxito si algún hijo tiene éxito (equivalente a un OR lógico)
- **Secuencia (Sequence):** ejecuta hijos en orden; devuelve Éxito solo si todos tienen éxito (equivalente a un AND lógico)
- **Decoradores:** modifican el comportamiento del hijo (ej: Inverter, Cooldown, Limit)
- **Condiciones:** evalúan una variable del Blackboard y devuelven Éxito o Fallo
- **Acciones:** ejecutan una tarea y pueden devolver Éxito, Fallo o En ejecución

El **Blackboard** actúa como memoria compartida donde todos los nodos leen y escriben variables, desacoplando las condiciones de las acciones y facilitando la depuración visual.

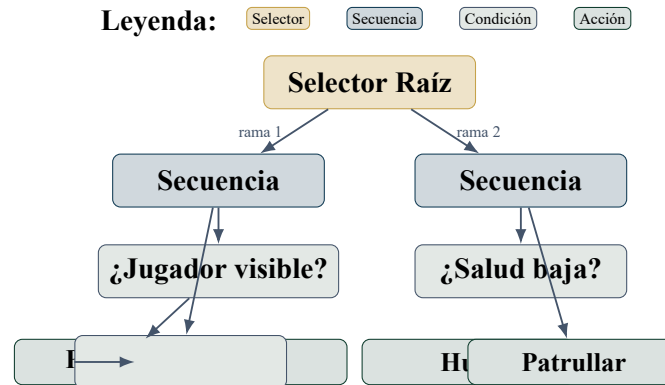


Figura 2: Estructura de un Árbol de Comportamiento. El Selector evalúa las ramas en orden; la primera que devuelve Éxito detiene la evaluación (cortocircuito).

2.2.2. Ventajas y limitaciones

Los BT son altamente escalables, modulares y fáciles de depurar visualmente (herramientas como Unreal Engine pintan de verde/rojo los nodos que tienen éxito/fallan). Su principal desventaja es que el tick desde la raíz cada frame puede ser costoso si el árbol es muy grande; la optimización clave son los *Event-Driven Behavior Trees*, que solo reevalúan ramas cuando cambian las variables del Blackboard.

2.2.3. Aplicación en Halo

Los **Grunts** y **Elites** utilizan Behavior Trees como núcleo de su IA. Los Grunts evalúan la variable `isEliteAlive` en un Selector de alta prioridad; cuando el Elite líder muere, el cortocircuito activa la rama de *Pánico*. Los Elites utilizan selectores anidados para priorizar la supervivencia sobre el ataque, evaluando primero la salud del escudo y luego la disponibilidad de cobertura.

2.3. Árboles de Decisión (Decision Trees)

2.3.1. Definición y fundamentos

Un **Árbol de Decisión** es un grafo dirigido donde cada nodo interno (un diamante en la representación visual) evalúa una condición binaria, y las ramas representan los resultados Sí o No, llevando finalmente a una hoja que dicta la acción a ejecutar. Están inspirados en los clasificadores CART (*Classification and Regression Trees*) del aprendizaje automático supervisado.

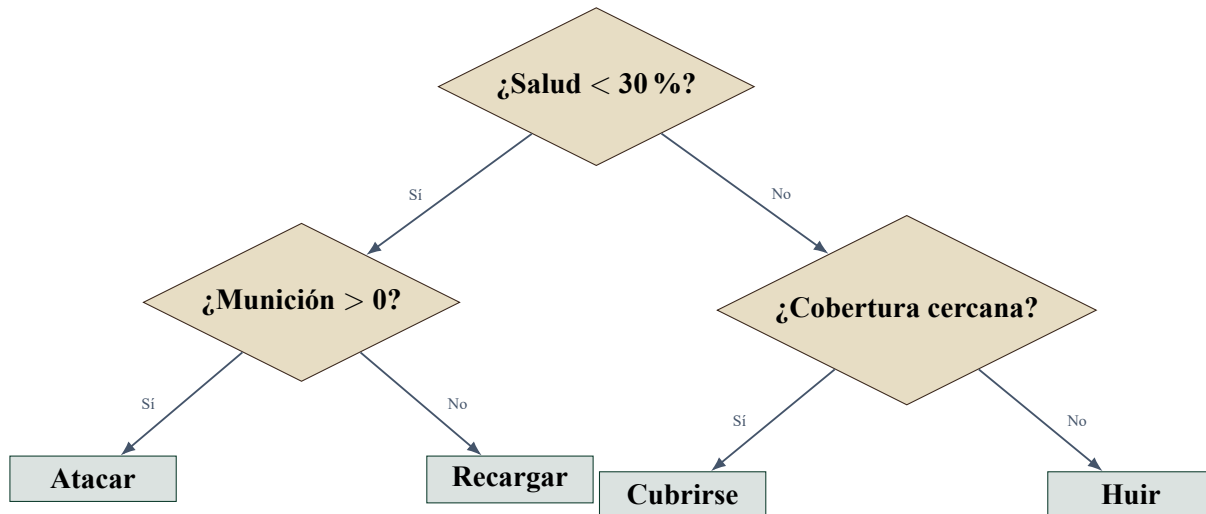


Figura 3: Árbol de Decisión para un NPC en combate. Cada nodo interno evalúa una condición binaria y las hojas determinan la acción.

2.3.2. Ventajas y limitaciones

Los DT son extremadamente rápidos a nivel de CPU y fáciles de entender. Sin embargo, son unidireccionales y sin memoria: no pueden *arrepentirse* de una decisión hasta que el ciclo de toma de decisiones vuelva a pasar por el nodo raíz. En la práctica, se combinan con FSM: el DT decide *qué* hacer (estrategia) y la FSM ejecuta *cómo* hacerlo (animaciones).

2.4. Sistemas Basados en Reglas

2.4.1. Definición y fundamentos

Los **Sistemas Basados en Reglas** utilizan una base de conocimiento compuesta por hechos y reglas de producción del tipo SI condición ENTONCES acción. Emplean encadenamiento hacia adelante (*forward chaining*): parten de los hechos actuales y aplican reglas hasta alcanzar una conclusión o acción.

El problema del conflicto —cuando múltiples reglas se disparan simultáneamente— se resuelve mediante:

- **Prioridad estática:** cada regla tiene un número asignado por el diseñador
- **Recencia:** los hechos más nuevos tienen prioridad
- **Especificidad:** la regla con más condiciones gana

2.4.2. Aplicación en videojuegos

El uso más conocido es el **Sistema Director** de *Left 4 Dead*, que utiliza reglas para puntuar el *estrés* de los jugadores. Si $\text{TiempoSinInfectados} > 5s$ Y $\text{SaludPromedio} > 80$, se dispara una oleada masiva. En *Halo*, el sistema **Encounter Logic** funciona de manera similar: inyecta variables globales que alteran las prioridades de todos los NPCs en el escenario actual.

2.5. GOAP (Goal-Oriented Action Planning)

2.5.1. Definición y fundamentos

GOAP es un sistema donde el programador no define el comportamiento explícito del agente, sino que le proporciona:

- Un conjunto de **metas** (estados deseados del mundo, ej: $\text{enemigoMuerto} = \text{true}$)
- Un conjunto de **acciones**, cada una con precondiciones, efectos y un coste numérico

El agente utiliza el algoritmo A^* para buscar en el *grafo de acciones* la secuencia de menor coste que transforme el estado actual en el estado meta. Este enfoque fue popularizado por *F.E.A.R.*, donde los soldados enemigos descubren tácticas emergentes como flanquear sin que el diseñador las haya programado explícitamente.

2.5.2. Limitaciones

El principal desafío es el alto coste computacional de la re-planificación constante. En *F.E.A.R.*, el plan se recalcula cada 0.5 segundos. Si el jugador se mueve mucho, el plan cambia constantemente y el agente parece indeciso. La solución es un umbral de tolerancia: solo se replanifica si el nuevo plan es al menos un 20 % mejor que el actual, evitando el *thrashing*.

2.6. Lógica Difusa (Fuzzy Logic)

2.6.1. Definición y fundamentos

A diferencia de la lógica booleana tradicional (valores 0 o 1), la **Lógica Difusa** trabaja con grados de verdad continuos en el intervalo $[0, 1]$, permitiendo definir variables lingüísticas como *un poco lejos*, *bastante herido* o *muy cerca*. Opera en tres etapas:

1. **Fuzzificación:** convertir valores numéricos (ej: $\text{distancia} = 15 \text{ m}$) en grados de pertenencia a conjuntos difusos (*Cerca*, *Medio*, *Lejos*)
2. **Inferencia:** aplicar reglas del tipo SI *Distancia es Cerca* Y *Salud es Baja* ENTONCES *IntensidadDeAtaque es Alta*

3. **Defuzzificación:** convertir el resultado difuso en un valor numérico de salida, usualmente mediante el cálculo del centroide

2.6.2. Aplicación

No se usa como técnica independiente, sino como módulo interno de Sistemas de Utilidad para modular curvas de puntuación. Por ejemplo, en *Los Sims*, el hambre no es un booleano sino un valor difuso que sube suavemente, haciendo que la transición a *Buscar Comida* sea orgánica.

2.7. Sistemas de Utilidad (Utility AI)

2.7.1. Definición y fundamentos

Un **Sistema de Utilidad** asigna una puntuación numérica a cada acción disponible basándose en el contexto actual, y selecciona la acción con la puntuación más alta. Matemáticamente, cada acción a tiene una función de utilidad $U(a) = f(\text{contexto})$, y la acción seleccionada es $\text{argmax}_{a \in A} U(a)$.

El verdadero arte está en el diseño de las **curvas de respuesta**. Se utilizan funciones sigmoideas, logarítmicas o exponenciales para evitar que una acción domine siempre. Por ejemplo:

$$U(\text{Atacar}) = \frac{1}{1 + e^{-k(\text{VidaEnemigo} - 50)}}$$

donde k controla la pendiente de la curva: un valor alto hace al agente más agresivo, uno más bajo más cauteloso.

2.7.2. Aplicación en Halo

Los **Elites** incorporan un sistema de utilidad implícito para priorizar la supervivencia. La función de utilidad de *Buscar Cobertura* se incrementa drásticamente cuando `shieldHealth` cae por debajo del 20 %, superando la utilidad de *Atacar* incluso cuando el jugador está en rango.

2.8. Aprendizaje por Refuerzo (Reinforcement Learning)

2.8.1. Definición y fundamentos

El **Aprendizaje por Refuerzo** es una rama del Machine Learning donde un agente aprende una política óptima π^* mediante interacción con el entorno, maximizando una señal de recompensa acumulada. Se formaliza como un Proceso de Decisión de Markov (MDP) (S, A, P, R, γ) :

- S : conjunto de estados del entorno
- A : conjunto de acciones disponibles

- $P(s' | s, a)$: probabilidad de transición al estado s' desde s con acción a
- $R(s, a)$: recompensa inmediata
- $\gamma \in [0, 1)$: factor de descuento para recompensas futuras

2.8.2. Realidad en la industria

En la industria del videojuego, el RL no se utiliza en tiempo real durante la ejecución del juego. Se emplea *offline* para entrenar bots que luego se exportan como redes neuronales de inferencia fija. Ejemplos incluyen *AlphaStar* (StarCraft II) y *OpenAI Five* (Dota 2). El principal desafío es el *reward shaping*: el agente tiende a explotar bugs del entorno para maximizar recompensa, por lo que las funciones de recompensa deben alinearse cuidadosamente con la diversión del jugador.

3. Comprensión Conceptual: Análisis Comparativo

3.1. Guía de selección de técnica

Tabla 1: Recomendación de técnica según el escenario de uso

Escenario	Técnica recomendada
Enemigo básico de tutorial (comportamiento predecible)	FSM
NPC con rutinas complejas y escalables (médico, comerciante)	Behavior Tree
IA que se adapta a cientos de variables sin planificar (estrategia 4X)	Utility AI
IA que <i>inventa</i> tácticas sorprendentes (FPS táctico)	GOAP
Suavizar curvas de detección o velocidad de giro	Fuzzy Logic (como módulo de Utility)
Oponente imbatible para pruebas de estrés (entrenamiento offline)	Reinforcement Learning
Director de flujo global de partida (ej: Left 4 Dead)	Rule-Based Systems
Clasificación rápida de información en tiempo real	Decision Trees

3.2. Análisis comparativo transversal

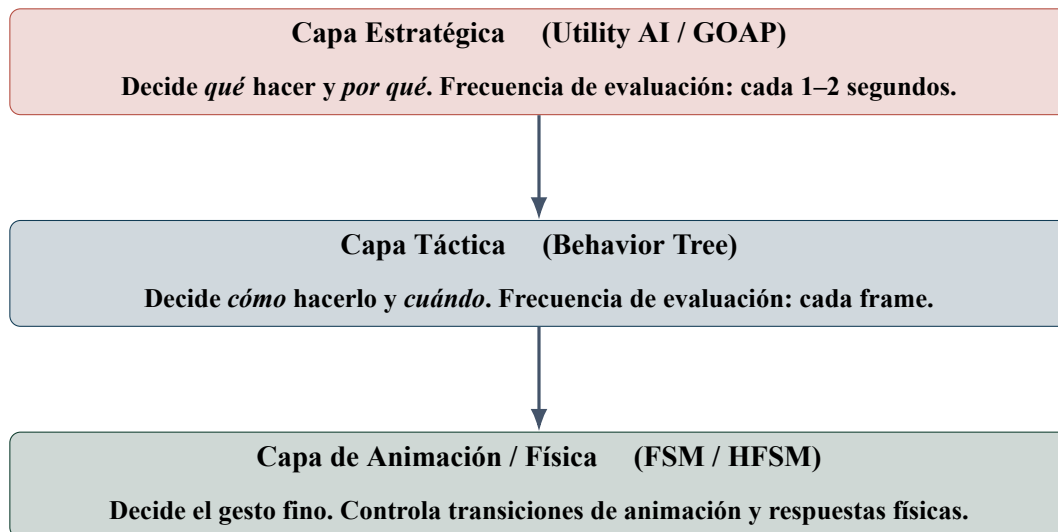
La siguiente tabla sitúa cada técnica en dos ejes fundamentales: **Reactividad** (capacidad de responder al instante) y **Planificación** (capacidad de pensar a futuro), junto con su predictibilidad y coste computacional.

Tabla 2: Comparativa de técnicas en reactividad, planificación, predictibilidad y coste

Técnica	Reactividad	Planificación	Predictibilidad	Coste CPU
FSM	Alta	Nula	Muy Alta	Bajísimo
Behavior Tree	Muy Alta	Baja	Alta	Bajo-Medio
Decision Tree	Alta	Nula	Muy Alta	Bajísimo
Rule-Based	Media	Media	Media	Medio
GOAP	Baja	Muy Alta	Baja	Muy Alto
Utility AI	Media-Alta	Baja	Media-Baja	Medio-Alto
Fuzzy Logic	Alta	Nula	Media	Bajo-Medio
RL	Baja	Alta	Muy Baja	Alto (GPU)

3.3. Arquitecturas Híbridas

Ningún juego AAA utiliza una sola técnica de forma aislada. La complementación real ocurre en tres capas bien diferenciadas, donde cada capa opera a una frecuencia distinta y se encarga de un nivel de abstracción diferente:



Ejemplo (*The Last of Us*): Utility elige objetivo → BT ejecuta flanqueo → HFSM mezcla animaciones

Figura 4: Arquitectura híbrida en tres capas utilizada en juegos AAA. Cada capa opera a una frecuencia distinta y resuelve un nivel de abstracción diferente.

4. Aplicación en Halo: Identificación de Técnicas por Personaje

En *Halo*, la IA de los NPCs utiliza principalmente **Árboles de Comportamiento** como núcleo de toma de decisiones, complementados con FSM para la capa de animación y sistemas basados en reglas para la lógica de escuadrón. La siguiente tabla identifica las técnicas aplicadas a cada personaje y las relaciona con acciones concretas observables durante el juego.

Tabla 3: Técnicas de toma de decisiones identificadas en personajes de Halo

Personaje	Técnica principal	Acción concreta en el juego
Grunt	Behavior Tree + FSM	Evalúa liderazgo del Elite; al morir el líder, activa rama de pánico: suelta arma, huye con animación torpe.
Elite	Behavior Tree + Utility	Evalúa salud de escudo; si $< 20\%$, prioriza cobertura sobre ataque; coordina escuadrón.
Jackal	Behavior Tree	Usa cobertura; dispara a distancia; se retira si el jugador se acerca demasiado.
Hunter	FSM + Behavior Tree	Patrulla en pareja; dispara ráfagas; carga si el jugador está en rango; expone punto débil dorsal.
Flood (infección)	FSM pura	Estados: merodear, detectar, cargar, atacar cuerpo a cuerpo; comportamiento de enjambre sin planificación.
Covenant (escuadrón)	Rule-Based + Blackboard	Comparten variables globales como refuerzos solicitados, estado del líder y prioridad de objetivo.

4.1. Análisis detallado: Grunts y Elites

4.1.1. Grunt (Infantería básica)

El árbol de comportamiento del Grunt evalúa en su nodo Selector de máxima prioridad la variable `isEliteAlive`. Cuando el Elite líder muere, el cortocircuito del selector detiene la evaluación de las ramas de menor prioridad (*Atacar*, *Buscar granadas*) y activa inmediatamente la rama de *Pánico*. Esta rama es una Secuencia que ejecuta en orden:

1. Soltar el arma principal (cambia el esqueleto de animación)
2. Activar animación de *huida torpe* (brazos en alto, carrera descoordinada)
3. Reproducir línea de audio de advertencia (*¡El jefe está muerto, corran!*)
4. Calcular vector de huida aleatorio opuesto a la última posición conocida del jugador

4.1.2. Elite (Rango superior)

El Elite incorpora un sistema de utilidad implícito en la evaluación de sus selectores. Cuando `shieldHealth` cae por debajo del 20 %, la utilidad de la acción *Buscar Cobertura* supera a la de *Atacar*. La rama seleccionada ejecuta:

1. Rodaje lateral con animación cinematográfica (`Dodge_Roll_Right`)
2. Movimiento al objeto etiquetado como *Cobertura* más cercano mediante navegación `Nav-Mesh`
3. Inicio de regeneración activa del escudo (el jugador ve cómo el indicador púrpura se recupera)
4. Comunicación al escuadrón: `BroadcastSignal("SuppressingFire!")` para que los Grunts cubran su retirada

5. Análisis de Funcionamiento: Flujo de Decisión en Halo

La IA de *Halo* ejecuta un ciclo continuo de cinco fases, evaluado entre 30 y 60 veces por segundo. Este flujo se implementa mediante Árboles de Comportamiento que gestionan desde la percepción del entorno hasta la actualización del estado interno del NPC.

5.1. Diagrama General del Flujo de Decisión

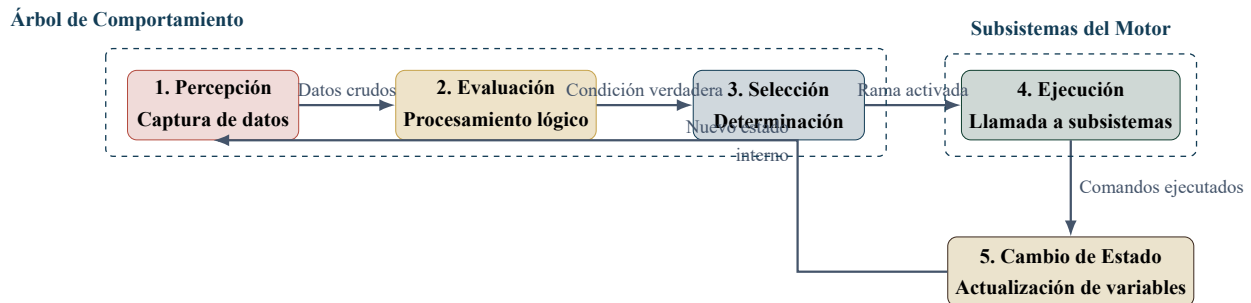


Figura 5: Ciclo completo de decisión de la IA en Halo. Las fases 1–3 ocurren en el Árbol de Comportamiento, la fase 4 en los subsistemas del motor y la fase 5 actualiza el Blackboard.

5.2. Fase 1: Percepción

El NPC no ve como un ser humano, sino que el motor del juego alimenta al nodo raíz del árbol con datos crudos obtenidos mediante sensores virtuales:

- **Raycasts (Visión):** conos de visión con detección de obstáculos. Cada NPC tiene un ángulo y alcance que determina su campo visual

- **Triggers de audio (Oído):** esferas de detección concéntricas que reaccionan a disparos, explosiones o pasos
- **Variables globales (Tacto/Conocimiento):** datos compartidos en el Blackboard como salud del líder, estado del escudo, posición del jugador o nivel de alerta del escuadrón

Ejemplo – Grunt: El sistema monitoriza `isEliteAlive`. Cuando el Elite muere, el sensor de *conocimiento de escuadrón* actualiza la variable a `false`. Simultáneamente, el sistema de audición confirma el origen del disparo que mató al líder, actualizando `lastKnownPlayerPosition` con la ubicación exacta.

Ejemplo – Elite: El sistema monitoriza `shieldHealth`. Al caer al 15 %, el `ThreatAnalyzer` escanea el entorno en busca de objetos con la etiqueta *Cover* dentro de un radio de 20 metros, utilizando el sistema de navegación (*NavMesh*) para validar que la cobertura sea alcanzable.

5.3. Fase 2: Evaluación de Condiciones

La información percibida fluye por la jerarquía del árbol de comportamiento. Los nodos de control evalúan las variables de arriba hacia abajo y de izquierda a derecha, siguiendo el principio de **cortocircuito**: en un Selector, si la primera condición es verdadera, se ignoran las condiciones restantes de esa rama.

Ejemplo – Grunt:

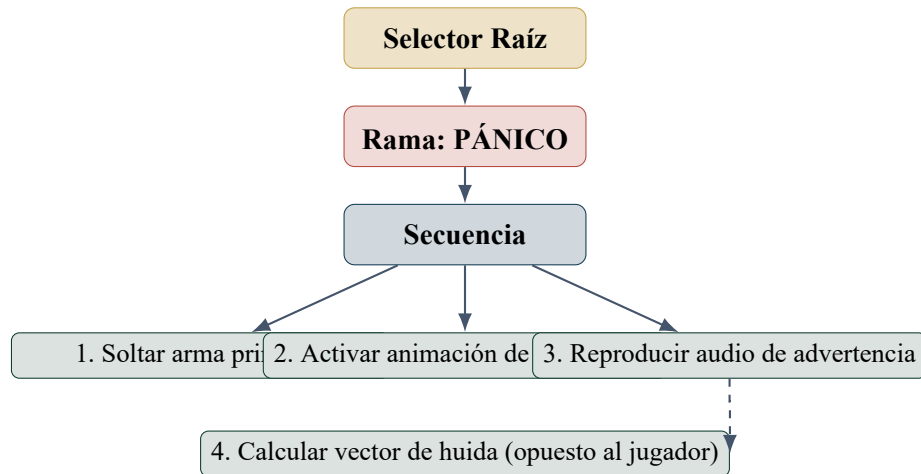
- **Condición A (alta prioridad):** *¿Está vivo mi líder?* → `Falso`
- **Resultado:** El Selector cortocircuita y detiene la evaluación de las ramas *Atacar* y *Buscar granadas*, activando inmediatamente la rama de *Pánico*

Ejemplo – Elite:

- **Condición A (supervivencia):** *¿Escudo < 20 %?* → `Verdadero`
- **Condición B (táctica):** *¿Cobertura disponible en radio de acción?* → `Verdadero`
- **Resultado:** El árbol selecciona la rama de *Búsqueda de Cobertura*, descartando acciones ofensivas como *Cargar contra el jugador* o *Lanzar granada*

5.4. Fase 3: Selección de Acción

Una vez que una condición se evalúa como verdadera, el árbol desciende hacia los nodos hoja que contienen las acciones concretas. Si la rama seleccionada es una Secuencia, las acciones deben ejecutarse en orden estricto para que la reacción sea creíble.



La Secuencia requiere que todas las acciones se ejecuten en orden para que la reacción sea creíble

Figura 6: Rama de Pánico del Grunt: secuencia de cuatro acciones en orden estricto. Si alguna falla, la secuencia completa devuelve Fallo.

5.5. Fase 4: Ejecución

El árbol de comportamiento no ejecuta las acciones por sí mismo; envía comandos a los subsistemas del motor gráfico, de audio y de navegación para materializar la decisión en pantalla. La ejecución es el momento en que la IA *toca* el mundo real del juego.

Tabla 4: Comandos enviados a los subsistemas durante la ejecución de las fases de Pánico (Grunt) y Cobertura (Elite)

Personaje	Subsistema	Comando	Acción concreta en pantalla
Grunt	Animación	Panic_Run	Suelta el arma, levanta brazos, corre descoordinado
Grunt	Audio	LeaderDown	“¡El jefe está muerto, corran!”
Grunt	Navegación	FleeDestination	Traza ruta de huida aleatoria
Elite	Animación	DodgeRoll_R	Rodaje lateral para esquivar disparos
Elite	Navegación	CoverMove	Se mueve a la cobertura más cercana
Elite	Escudo	ShieldRegenOn	Inicia recarga activa del escudo
Elite	Escuadrón	SuppressFire	Grunts disparan para cubrir su retirada

5.6. Fase 5: Cambio de Estado

La ejecución finaliza alterando el estado interno del NPC y su Blackboard. Este nuevo estado determinará cómo se comportará en el siguiente ciclo de evaluación, alterando sus prioridades futuras y permitiendo que la IA *recuerde* lo que acaba de hacer.

Ejemplo – Grunt (de combatiente a fugitivo):

- **Estado interno:** COMBATE_TÁCTICO → HUIDA

- **Blackboard:** `isPanicActive = true`, `panicTimer = 5.0s`, `ignoreCombatOrders = true`
- **Efecto:** En los siguientes ticks, el Grunt ignora eventos como *jugador a la vista*. Su árbol evita todas las ramas de ataque y solo reevalúa la rama de huida hasta que el temporizador expire o esté lejos del peligro

Ejemplo – Elite (de ofensivo a defensivo):

- **Estado interno:** `ATAQUE_ACTIVADO` → `COBERTURA_DEFENSIVA`
- **Blackboard:** `shieldRegenerating = true`, `holdPositionUntilShieldFull = true`
- **Efecto:** Mientras está en `COBERTURA_DEFENSIVA`, el árbol no evalúa las ramas de *Carga Berserker* ni *Flanqueo*. Solo monitorea el nodo de *Espera* que observa la variable `shieldHealth`. Al llegar al 100 %, el estado vuelve a `ATAQUE_ACTIVADO`

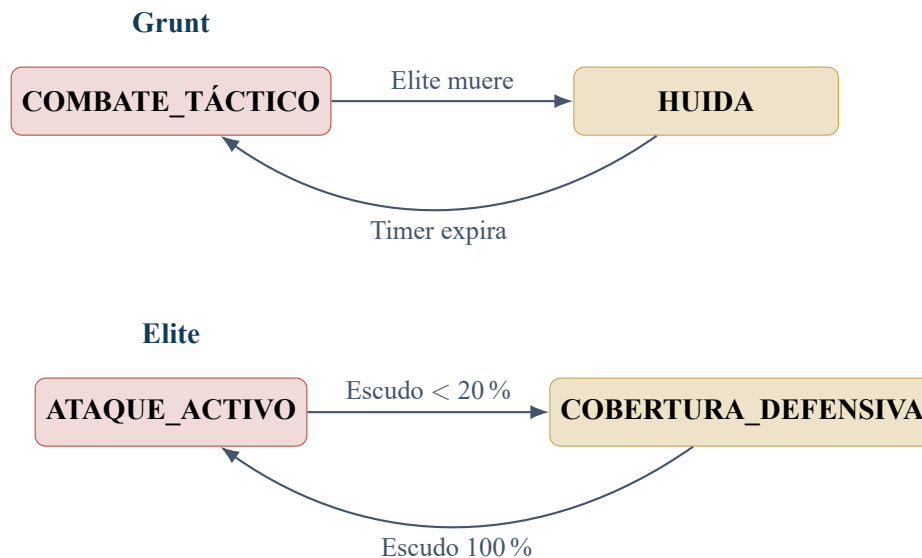


Figura 7: Transiciones de estado interno para Grunt y Elite en Halo. Cada transición modifica el Blackboard y altera las prioridades del árbol en el siguiente ciclo.

6. Conclusiones

La investigación realizada demuestra que la IA de *Halo* implementa un sistema híbrido de toma de decisiones que combina múltiples técnicas de forma jerárquica y complementaria. El núcleo del sistema son los **Árboles de Comportamiento**, que proporcionan la estructura modular y escalable necesaria para gestionar la complejidad de los distintos NPCs. Esta arquitectura se complementa con:

- **FSM** para la capa de animación y para personajes simples como los Flood
- **Sistemas de Utilidad** implícitos en los selectores de los Elites para priorizar supervivencia

sobre ataque

- **Sistemas Basados en Reglas** en la lógica de escuadrón y en el sistema global de *Encounter Logic*

El flujo de decisión en cinco fases (percepción, evaluación, selección, ejecución y cambio de estado) permite que cada NPC reaccione de forma creíble y contextual:

- Los **Grunts** reaccionan al caos y al miedo de forma instantánea mediante cortocircuito en selectores, demostrando cómo un diseño simple puede generar comportamientos convincentes
- Los **Elites** toman decisiones tácticas basadas en su supervivencia, priorizando cobertura sobre ataque y coordinando al escuadrón mediante comunicación implícita
- El sistema global de **Encounter Logic** inyecta variables que alteran el flujo de todos los NPCs simultáneamente, adaptando la dificultad y el ritmo del combate

El resultado no es una IA que simplemente *gana*, sino una que mantiene la ilusión de vida y la diversión táctica en cada combate. La comprensión de estas técnicas y su aplicación práctica en un videojuego comercial permite apreciar cómo la arquitectura de IA debe balancear cuidadosamente reactividad, planificación y coste computacional. Como regla general: cuanto más complejo y escalable deba ser el comportamiento, más arriba en la cadena de técnicas (FSM → BT → GOAP/Utility) debe situarse la implementación, pero siempre considerando que la técnica más sofisticada no siempre es la mejor para cada personaje.